

Resolución de problemas y algoritmos

Dra. Jessica Andrea Carballido

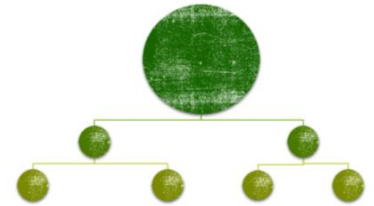
jessicarballido@gmail.com



Dpto. de Ciencias e Ingeniería de la Computación

UNIVERSIDAD NACIONAL DEL SUR

DESCOMPOSICIÓN DE PROBLEMAS



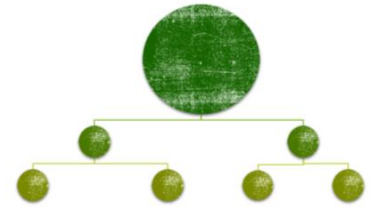
Cuando la complejidad de los problemas aumenta, la tarea de hallar una solución se torna más difícil.

Una metodología para **reducir la complejidad** consiste en plantear la solución del problema a partir de la solución de una serie de subproblemas más sencillos que forman parte del problema original.

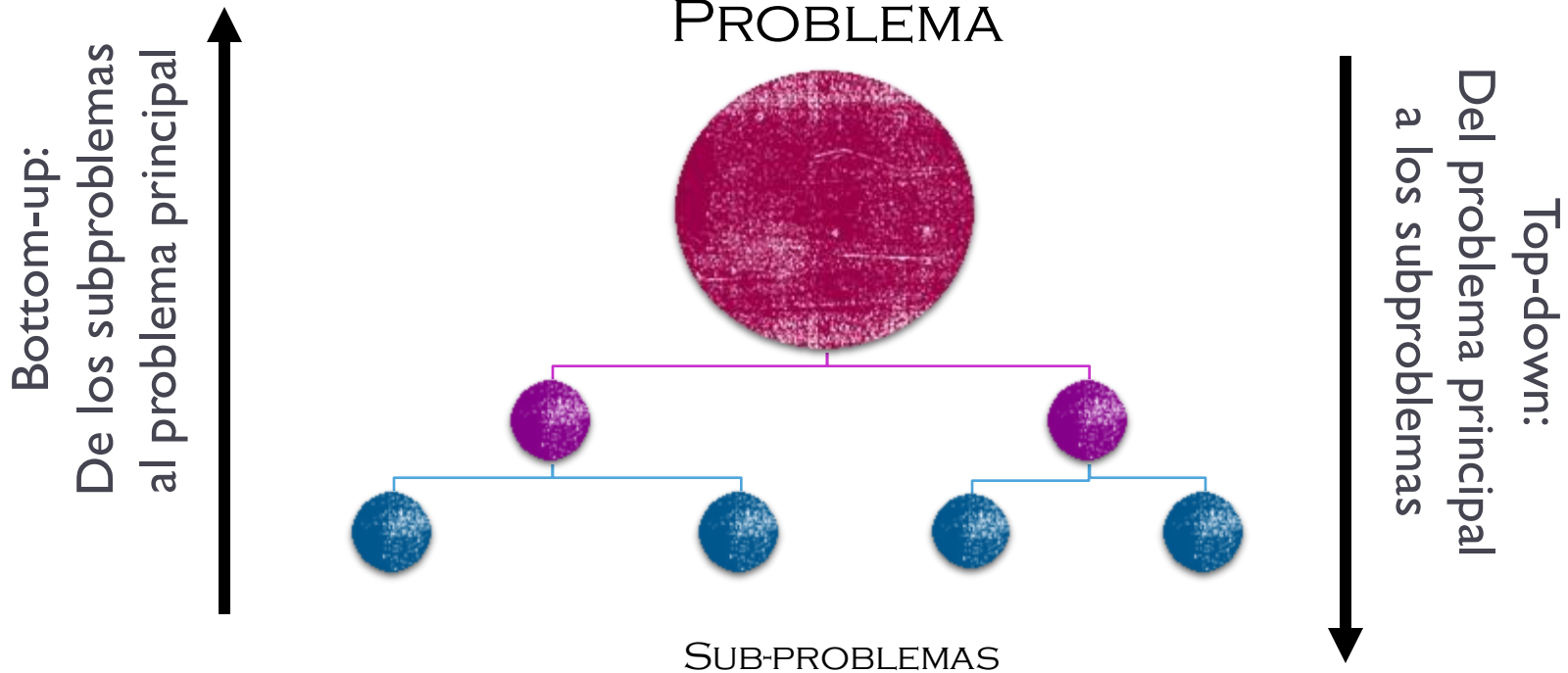
La **estrategia de diseño dividir para conquistar** consiste en dividir un problema complejo en dos o más sub-problemas más simples. Cada sub-problema puede resolverse directamente o volver a dividirse en sub-problemas más simples.

La estrategia se complementa con el **refinamiento paso a paso**. Las soluciones de los sub-problemas se integran para resolver el problema original.

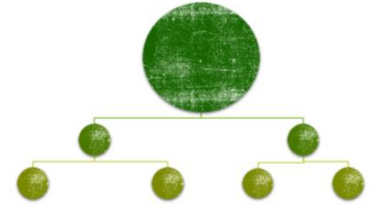
DESCOMPOSICIÓN DE PROBLEMAS



“BOTTOM-UP VS. TOP-DOWN”



DESCOMPOSICIÓN DE PROBLEMAS



- # La técnica *bottom-up* consiste en ir resolviendo en primer lugar los problemas más elementales, usando esas soluciones para resolver problemas mayores.
- # La técnica *top-down* resuelve primeramente el problema mayor, dejando pendiente la solución de los problemas más elementales para más adelante.

DESCOMPOSICIÓN DE PROBLEMAS



La **programación modular** es una metodología de programación que consiste en organizar un programa en *módulos*.

En la etapa de diseño de un programa se aplica la estrategia de **Dividir para Conquistar**.

En la etapa de **implementación**, cada uno de los sub-problemas se implementa a través de un **módulo**.

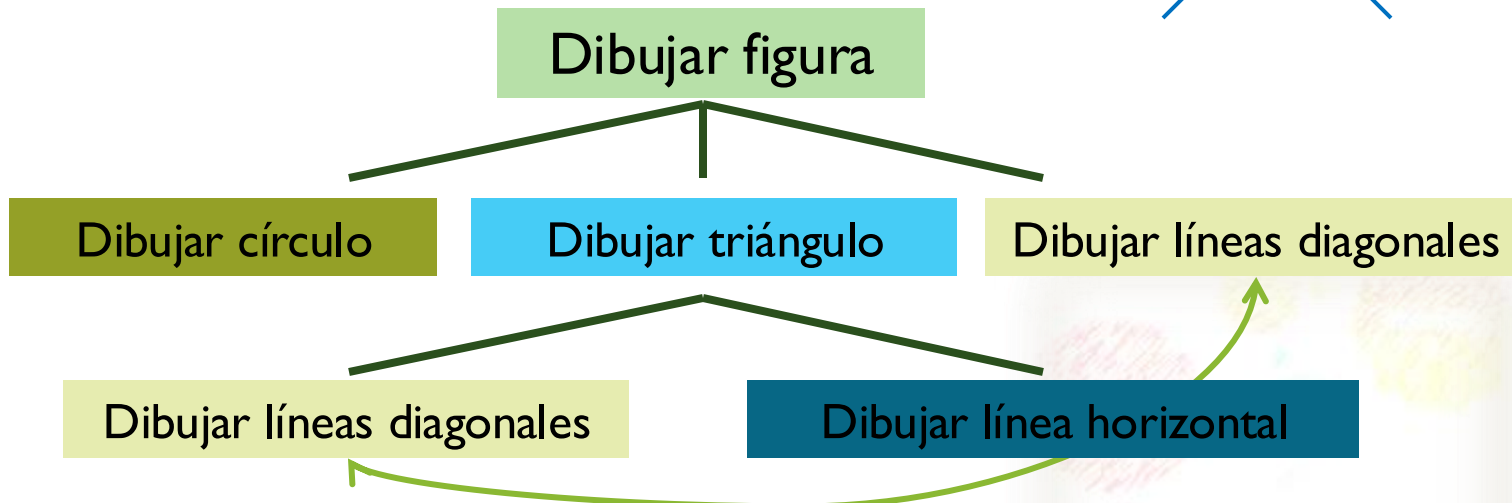
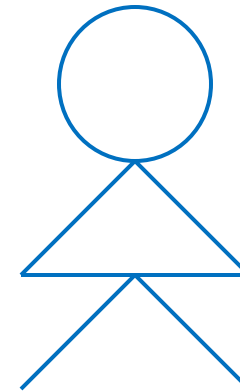
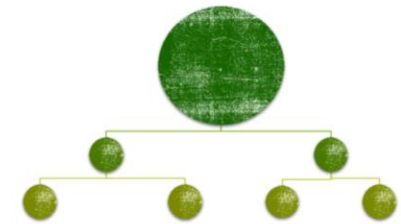
Los lenguajes de programación brindan diferentes mecanismos para implementar módulos.

En Pascal los *módulos* más simples son los procedimientos y las funciones.

Descomponer un problema

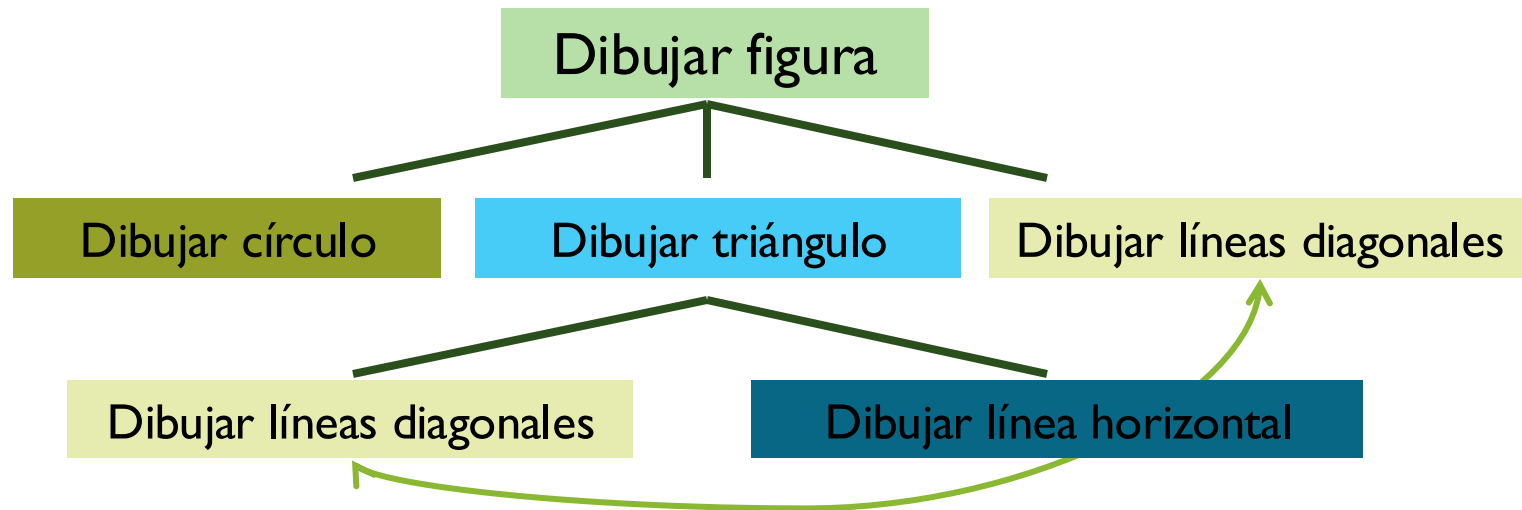
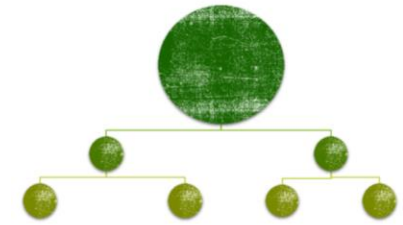
Problema: Dibujar esta figura

1. Dibujar el círculo
2. Dibujar el triángulo
 1. Dibujar las dos líneas diagonales
 2. Dibujar la línea horizontal
3. Dibujar otras dos líneas diagonales



En realidad resuelven el mismo “sub-problema”

Descomponer un problema



En realidad resuelven el mismo “sub-problema”

Dibujar círculo, dibujar líneas diagonales y dibujar línea horizontal son acciones **primitivas** que se implementan como subprogramas.

```

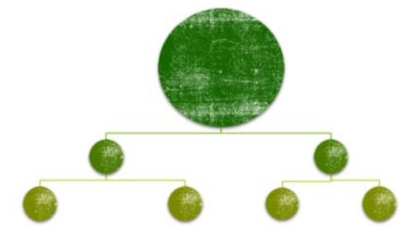
program dibujo;
  procedure dibujaCirculo;
  begin
    writeln('  * ');
    writeln(' * * ');
    writeln('  * ');

  end;
  procedure dibujaDiagonales;
  begin
    writeln(' \ ');
    writeln(' / \ ');
    writeln('/   \ ');

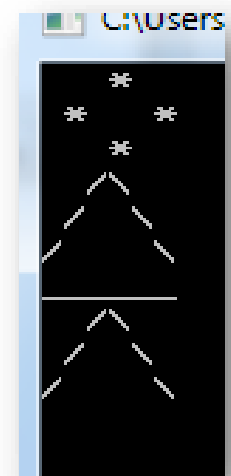
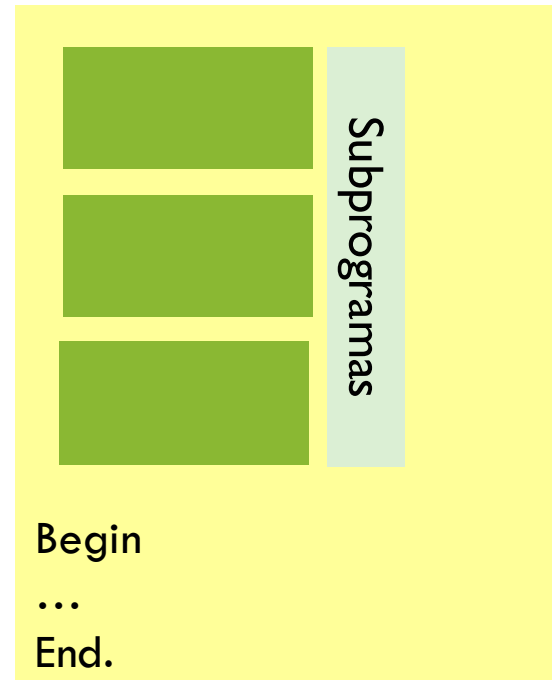
  end;
  procedure dibujaLinea;
  begin
    writeln('_____');

  end;
begin
  dibujaCirculo;  dibujaDiagonales;  dibujaLinea;  dibujaDiagonales;
  readln;
end.

```



Programa Principal





Los sub-programas (primitivas)

Bloques de código que llevan a cabo una tarea concreta
(resuelven un **sub-problema** específico)

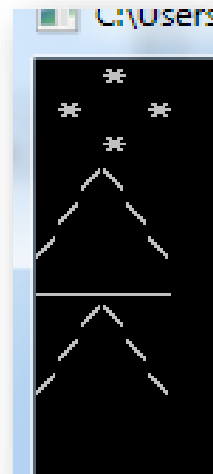
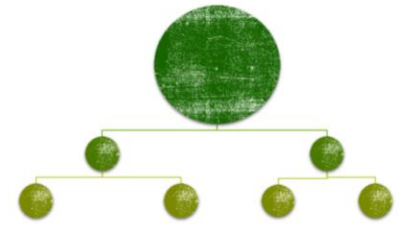
Se “crea” una nueva **primitiva** (**sub-programa**)

- Tienen un **propósito** bien definido.
- Permiten **reutilizar código** de manera sencilla y segura
 - Pueden ser usados más de una vez en el programa principal sin necesidad de reescribir todo (o *copiar-pegar*).
- Ayudan a que el código del programa principal sea:
 - **Más Legible**: Resulta más sencillo leer sólo el nombre del subprograma que todo su código.
 - **Más Ordenado**: Cada subprograma ocupa un lugar concreto dentro de todo el código.

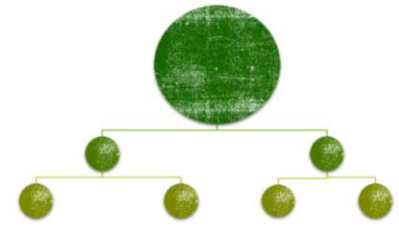
```
program dibujoSinPRIMITIVAS;  
begin
```

```
    writeln(' * ');  
    writeln(' * * ');  
    writeln(' * ');  
    writeln(' ^ ');  
    writeln(' / \ ');  
    writeln('/   \ ');  
    writeln('_____');  
    writeln(' ^ ');  
    writeln(' / \ ');  
    writeln('/   \ ');
```

```
end.
```



Los sub-programas



Hay que distinguir entre la declaración y la invocación de sub-programas.

- ✓ La **declaración** sirve para **definir** el subprograma
- ✓ La **invocación** sirve para **usar** el subprograma dentro del programa principal (o de otro subprograma).

Hasta ahora, **habíamos invocado**
procedimientos y funciones predefinidas.
¿Cuáles?

Hoy vamos a aprender a definir nuestras primitivas.

```
program dibujo;
```

```
  procedure dibujaCirculo;
```

```
  begin
```

```
    writeln('  *  ');
```

```
    writeln(' * * ');
```

```
    writeln('  *  ');
```

```
  end;
```

```
  procedure dibujaDiagonales;
```

```
  begin
```

```
    writeln('  \  ');
```

```
    writeln(' /  \ ');
```

```
    writeln('/    \');
```

```
  end;
```

```
  procedure dibujaLinea;
```

```
  begin
```

```
    writeln('_____');
```

```
  end;
```

```
begin
```

```
  dibujaCirculo;
```

```
  dibujaDiagonales;
```

```
  dibujaLinea;
```

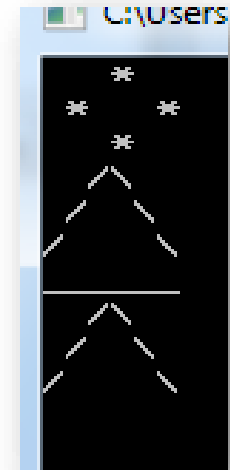
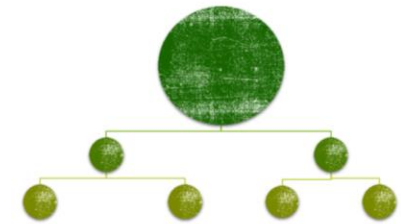
```
  dibujaDiagonales;
```

```
  readln;
```

```
end.
```

Declaraciones

Invocaciones



```

program dibujo;
var i: integer;
procedure dibujaCirculo;
begin
    writeln('  *  ');
    writeln(' * * ');
    writeln('  *  ');
end;
procedure dibujaDiagonales;
begin
    writeln('  \  ');
    writeln(' / \ ');
    writeln('/   \');
end;
procedure dibujaLinea;
begin
    writeln('_____');
end;
begin
    ...
    readln;
end.

```

Y cómo quedaría este programa sin primitivas?

Declaraciones

Invocaciones

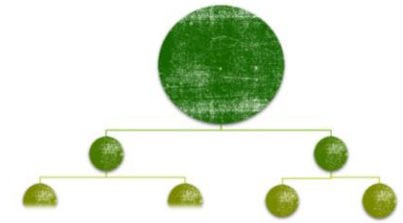
```

for i:=1 to 3 do
begin
    dibujaCirculo;
    dibujaCirculo;
    dibujaDiagonales;
    dibujaLinea;
end;

```



Los sub-programas



En Pascal, los sub-programas (primitivas) son implementados mediante **procedimientos** o **funciones**.

- Procedimientos predefinidos: *write*, *writeln*, *read*, *readln*
- Funciones predefinidas: *sqr*, *abs*, *chr*, *ord*, *eof*, *eoln*

FUNCION

La invocación se realiza desde una **expresión**.

En general constituye un cálculo numérico o la determinación de un valor de verdad

```
Program p;  
var n: integer;  
begin  
  writeln('Ingrese un número:');  
  readln(n);  
  if (abs(n)=n)  
  then  
    writeln('El numero es positivo.')  else  
    writeln('El numero es negativo.')end.
```

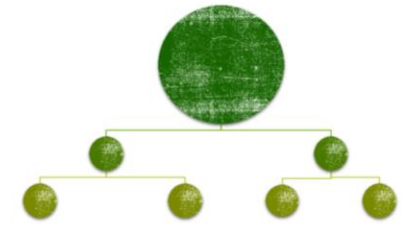
PROCEDIMIENTO

La invocación constituye en sí misma una **instrucción**.



Algunas funciones predefinidas de Pascal:

Abs(x)	Proporciona el valor absoluto de una variable numerica x.
ArcTan(x)	El arco cuya tangente es x.
Chr(x)	Devuelve el carácter ASCII de un entero entre 0 y 255.
Cos(x)	Proporciona el valor del coseno de x.
Exp(x)	La exponencial de x(e ^X).
Frac(x)	Parte decimal de x.
Int(x)	Parte entera de x.
Ln(x)	Logaritmo neperiano de x.
Odd(x)	True si x es impar, y false si es par.
Ord(x)	Ordinal de una variable tipo ordinal x.
Pred(x)	Ordinal anterior a la variable ordinal x.
Round(x)	Entero más próximo al valor x.
Succ(x)	Ordinal siguiente a la variable ordinal x.
Sin(x)	Seno de x.
Sqr(x)	Cuadrado de x.
Sqrt(x)	Raiz cuadrada de x, para x>=0.
Trunc(x)	Parte entera de x.

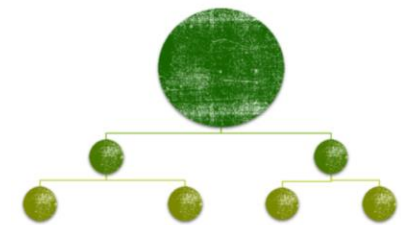


Se podrá hacer....

```
n:=sqr(round(r1)*round(r2));
```

```
if (sqrt(trunc(r)+100)*2>0)  
then....
```

Algunos procedimientos predefinidos de Pascal:



write	Escribe en pantalla o en un archivo
read	Lee del buffer o de un archivo
writeln	Escribe en pantalla o en un archivo de texto
readln	Lee del buffer o de un archivo de texto
assign	Vincula un nombre lógico con un nombre físico de archivo
reset	Abre un archivo para lectura
rewrite	Crea un archivo vacío y lo deja preparado para escritura
close	Cierra un archivo

TRAZA

```
n1: -  
n2: ?  
Ingrese un numero real: 110.25  
r2:  
ch1: ?
```

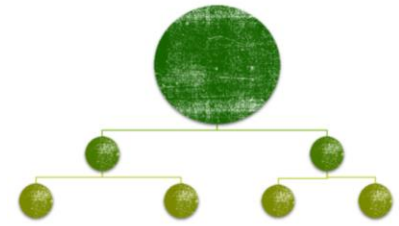
```
program fyp;
```

```
var n1, n2: integer;  
    r1, r2: real;  
    ch1: char;
```

```
begin
```

```
n1:= 10 + round(3.5) + 2345 mod 100;  
n2:= trunc(2.8) * sqr(5) + ord('Z');  
writeln('n1: ', n1);  
writeln('n2: ', n2);  
write('Ingrese un numero real: ');  
readln(r1);  
r2:=r1/n1;  
writeln('r2: ', r2:2:2);  
if (sqrt(r2)<10)  
    then  
        ch1:=chr(n1)  
    else  
        ch1:=chr(pred(n2));  
writeln('ch1: ', ch1);  
readln; readln;  
end.
```


Procedimientos y Funciones

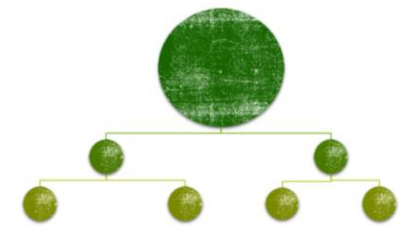


Un programa en Pascal puede incluir procedimientos y funciones **definidos por el programador.**

En la **declaración** se establece su **nombre** y la lista de **parámetros** (**datos** que se intercambian con el exterior).

Cuando un procedimiento o función ha sido declarado puede ser **usado** mediante una instrucción de **invocación**.

Datos en un sub-programa



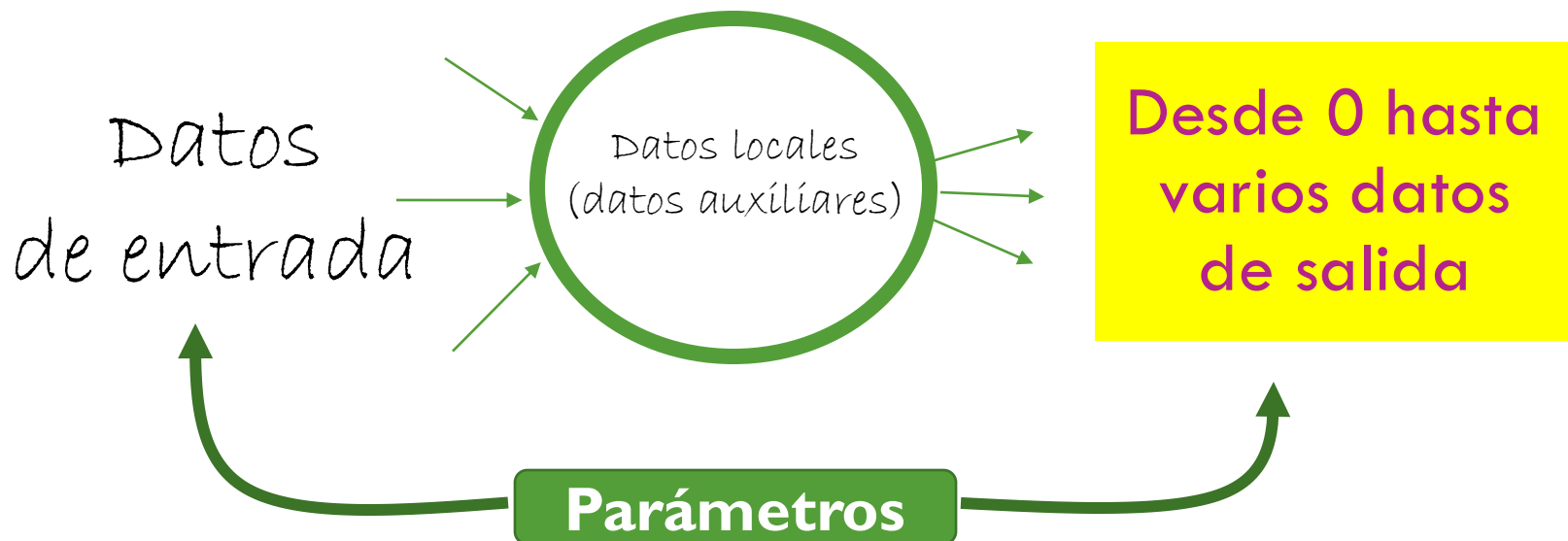
Datos locales (datos auxiliares del algoritmo)

- Variables y constantes declaradas y usadas solamente **dentro del subprograma**

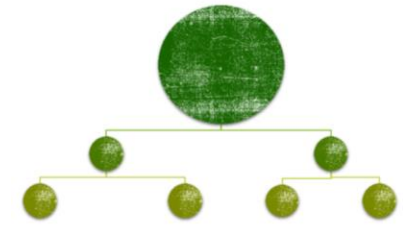
PROCEDIMIENTO

Datos intercambiados **con el exterior (parámetros)**

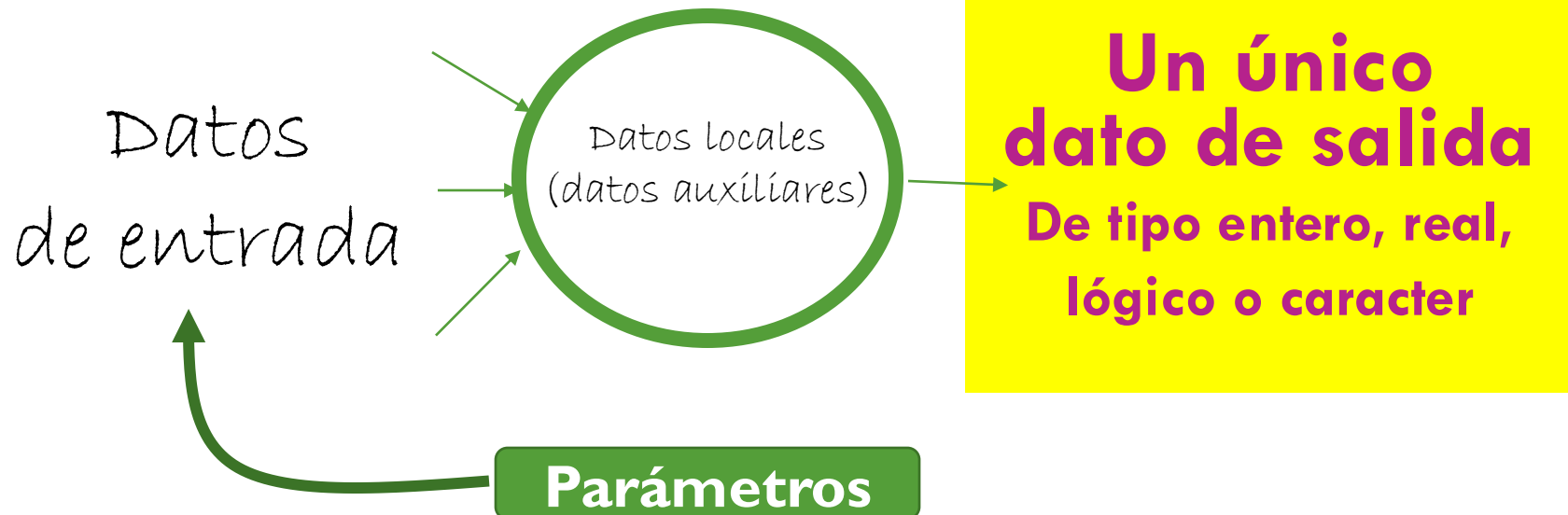
- Datos de entrada y datos de salida



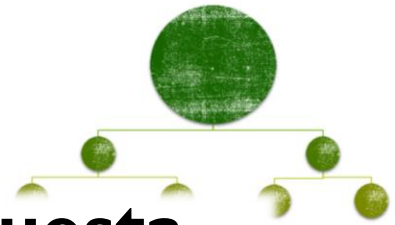
Funciones



FUNCIÓN: sub-programa con un único dato de salida de tipo simple, cuyo resultado es utilizado en el contexto de una expresión.



Funciones



Una **función** es una **instrucción compuesta** que tiene un **nombre**, puede tener **parámetros**, puede contener **declaraciones** y **debe retornar un único valor** de un tipo simple.

Algoritmo **minimo**

DE: n, m {enteros}

DS: **minimo** {entero}

Comienzo

Si $(n < m)$

entonces

$\text{minimo} \leftarrow n$

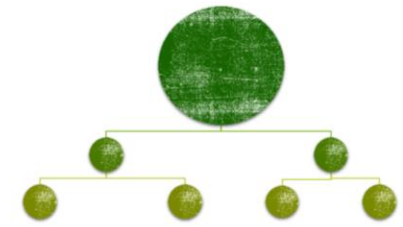
sino

$\text{minimo} \leftarrow m$

Fin



Funciones



- El **nombre** la función es **minimo**.
- Los **parámetros** de la función (datos de entrada) son dos enteros **n** y **m**.
- El **resultado** (dato de salida) es de tipo *integer* y **se liga a través de una asignación usando el nombre de la función**.

**Bloque
ejecutable
de la función**

```
function(minimo)(n, m: integer): integer;  
begin  
  if (n < m)  
  then minimo := n  
  else minimo := m  
end;
```

*Representa
al dato
de salida*

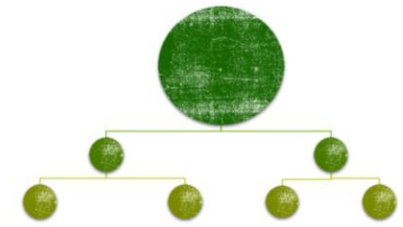
Siempre tiene que haber al menos una asignación
donde se establece el valor del DS = nombre de la función



**Al llegar al “end;”
del bloque ejecutable de la función,
el nombre de la función
debe tener asignado
el resultado de la misma.**

**Para todos los posibles casos,
debe devolver un resultado.**

Funciones



```
program pruebaMin;  
var n1, n2, min: integer;  
function minimo (n, m: integer): integer;  
begin
```

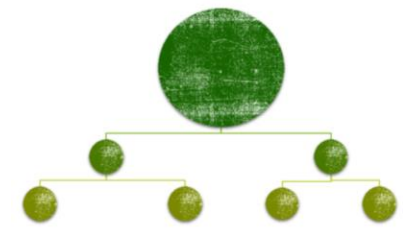
Parámetros
formales

```
    if (n < m)  
    then minimo := n  
    else minimo := m  
end;  
begin  
    ...  
end.
```

Dentro del **bloque ejecutable de la función**, cuando en la parte izquierda de una asignación aparece el nombre de la función, estamos indicando cuál va a ser el **resultado** de computar la función.

- n y m son los **parámetros formales** de la función, aparecen en la declaración de la función.
- EN LA DEFINICION DE LA FUNCION, el nombre funciona “como una variable” del tipo de la función que se usa SOLAMENTE del lado izquierdo de una asignación (por ahora) para almacenar el dato de salida.

Funciones



En la
declaración

Parámetros
formales

```
program pruebaMin;  
var n1, n2, min: integer;  
function minimo (n, m: integer): integer;
```

Bloque ejecutable

```
begin  
...  
end;
```

En la
invocación

Parámetros
reales

```
begin  
...  
min := minimo(n1, n2);  
end.
```

En el bloque ejecutable del programa principal
invocamos a la función a través de su nombre.

n1 y n2 son los **parámetros reales** de la función.

Los parámetros formales y reales
tienen que coincidir en **tipo, orden y número**.

Funciones

```
program pruebaMin;  
var n1, n2, min: integer;
```

```
function minimo(n , m: integer): integer;  
begin  
  if (n < m)  
  then minimo := n  
  else minimo := m  
end;
```

```
begin  
  write('Ingrese dos números ');  
  readln(n1, n2);  
  min := minimo(n1, n2);  
  writeln ('El minimo es ',min);  
end.
```

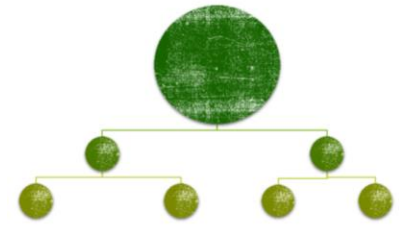


Por el momento, el **nombre de la función** solo podrá aparecer del lado izquierdo de una asignación
(en su bloque ejecutable).

} writeln('El minimo es ', minimo(n1, n2));

Funciones

```
program pruebaMin;  
var n1, n2, n3, min: integer;  
function minimo(n, m: integer): integer;  
begin  
  if (n < m)  
  then minimo := n  
  else minimo := m  
end;  
begin  
  write ('Ingrese tres números ');  
  readln (n1, n2, n3);  
  writeln('El minimo es ', minimo(minimo(n1, n2), n3))  
end.
```



¿Y si quiero ver el mínimo de 3 números?
¿Puedo usar la misma función?

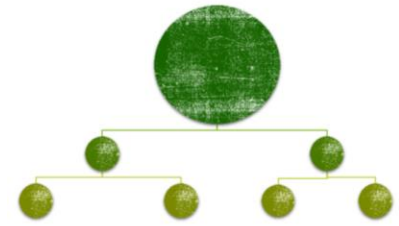


Funciones

```
program pruebaMin;  
var a1, a2, b3, b4, minA, minB, min: integer;
```

```
function minimo(n, m: integer): integer;  
begin  
  if (n < m)  
  then minimo := n  
  else minimo := m  
end;
```

```
begin  
  write ('Ingrese CUATRO números: ');  
  readln (a1, a2, b1, b2);  
  // obtengo el menor entre a1 y a2  
  minA := mínimo(a1, a2);  
  // obtengo el menor entre b1 y b2  
  minB := mínimo(b1, b2);  
  min := mínimo(minA, minB);  
  writeln('El mínimo entre los 4 numeros es ', min)  
end.
```



¿Y si quiero el mínimo de 4 números?
¿Puedo usar la misma función?





PROCEDIMIENTOS

Pueden tener desde 0 hasta varios datos de salida.



```
procedure NOMBRE(.....);
```

```
procedure NOMBRE(datos de entrada y de salida);  
var {variables locales};  
begin  
...  
end;
```

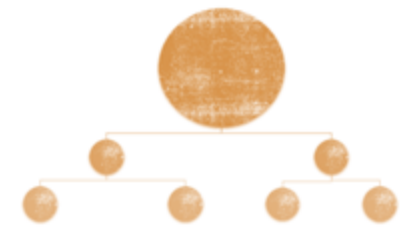

Procedimientos

Un **procedimiento** es una **instrucción compuesta** que tiene un **nombre**, *puede* contener **declaraciones**, **se comunica con su entorno a través de parámetros** y es invocado por una **instrucción**.

Pascal incluye algunos procedimientos predefinidos como *read, readln, write, writeln, assign, reset, rewrite*, etc.

El programador puede definir nuevos procedimientos y usarlos tal como si hubieran sido provistos por el lenguaje.

Puede tener desde 0 hasta cualquier cantidad de datos de salida.



```
program dibujo;  
  procedure dibujaCirculo;  
  begin  
    writeln(' * ');  
    writeln(' * * ');  
    writeln(' * * ');  
  end;  
  procedure dibujaDiagonales;  
  begin  
    writeln(' ^ ');  
    writeln(' / \ ');  
    writeln('/   \ ');  
  end;  
  procedure dibujaLinea;  
  begin  
    writeln('_____');  
  end;  
begin  
  dibujaCirculo;  dibujaDiagonales;  dibujaLinea;  dibujaDiagonales;  
  readln;  
end.
```

Las tres primitivas:

- No tienen datos de entrada.
- No tienen datos de salida.

Procedimientos



En el ejemplo anterior los procedimientos no tenían parámetros.

Problema: Escribir una línea divisoria con N guiones.

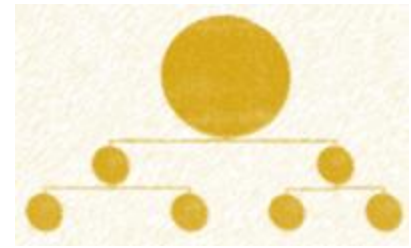
```
procedure linea(guiones: integer);  
  var i: integer;  
  begin  
    for i := 1 to guiones do  
      write("_");  
    end;
```

- Un dato de entrada.
- Sin datos de salida.



Procedimientos

Dato de entrada =
parámetro por valor

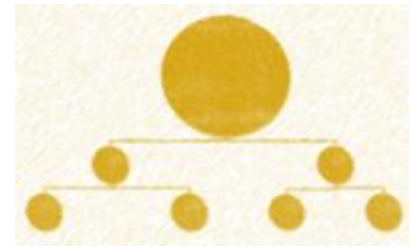


```
program P;  
var n: integer;  
procedure linea(guiones: integer);  
var i: integer;  
begin  
  for i:=1 to guiones do  
    write("_");  
  end;  
begin  
  writeln('Ingrese la cantidad de guiones de las líneas: ');  
  readln(n);  
  linea(n);  
  linea(n);  
end.
```

Variable
local

Se obtiene el valor de **n**
(parámetro real)
y ese valor es asignado a **guiones**
(parámetro formal).

Procedimientos



Problema: Dada una secuencia de números naturales (y su longitud) se desea mostrar, para cada número de la secuencia, la suma de sus dígitos y el producto de sus dígitos.

Problema principal: Recorrer completa una secuencia de números, dada su longitud.

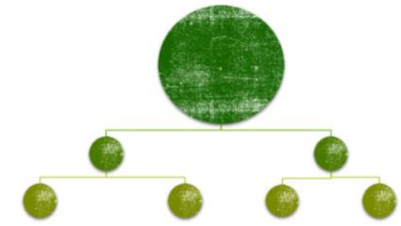
Sub-problema: Dado un número, calcular la suma y el producto de sus dígitos.

Algoritmo RecorridoSec

DE: LONG, secuencia de números naturales

DS: cartel con suma y producto de cada número

Daux: i, numero (natural)



Para i desde 1 hasta LONG hacer

 leer(numero)

 calcular la suma y el producto de los dígitos del numero

 mostrar la suma y el producto



Esto será nuestro PROGRAMA
PRINCIPAL

Algoritmo SumayProd

DE: N (natural)

DS: sumaDigs, prodDigs (natural)

sumaDigs $\leftarrow$ 0, prodDigs $\leftarrow$ 1

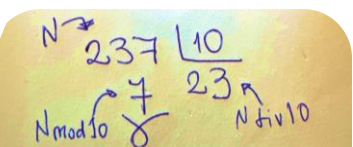
Mientras (N > 0) hacer

 sumaDigs $\leftarrow$ sumaDigs + (N mod 10)

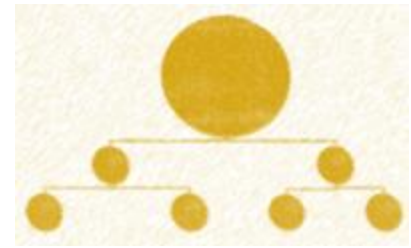
 prodDigs $\leftarrow$ prodDigs * (N mod 10)

 N $\leftarrow$ N div 10

Como tenemos DOS DATOS DE SALIDA,
lo vamos a implementar con un



Procedimientos



Dato de entrada

Datos de salida

```
Procedure SumayProd(N: integer; var sumaDigs, prodDigs: integer);
```

```
Begin
```

```
sumaDigs := 0; prodDigs := 1;
```

```
While (N > 0) do
```

```
    begin
```

```
        sumaDigs := sumaDigs + N mod 10;
```

```
        prodDigs := prodDigs * (N mod 10); // poner paréntesis, ojo precedencia
```

```
        N := N div 10;
```

```
    end;
```

```
End;
```

Bloque ejecutable del
procedimiento

PROGRAM secLong;

Var i, numero, LONG, SD, PD: integer;

Procedure SumayProd(N: integer; **var** sumaDigs, prodDigs: integer);

Begin

sumaDigs := 0; prodDigs := 1;

While (N > 0) do

begin

sumaDigs := sumaDigs + N mod 10;

prodDigs := prodDigs * (N mod 10); // poner paréntesis, ojo precedencia

N := N div 10;

end;

End;

Bloque ejecutable del procedimiento

Begin

Writeln('Ingrese la longitud de la secuencia'); read(LONG);

Writeln('Ingrese la secuencia de números naturales');

For i := 1 to LONG do

Begin

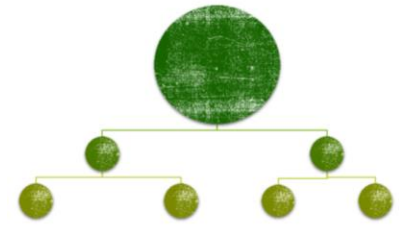
read(numero);

SumayProd(numero, SD, PD);

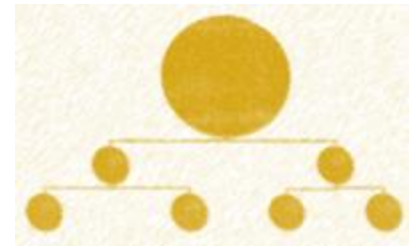
writeln('Para el número ', numero, ' la suma de sus dígitos es ', SD, y el producto de sus dígitos es ', PD)

End;

End.



Procedimientos



Primitiva P

DE: n (entero), m (real)

DS: a (entero), b y c (reales), d (lógico)



Cuatro datos de salida

Procedure P (n: integer; m: real; **var** a: integer; **var** b, c: real;
var d: boolean);

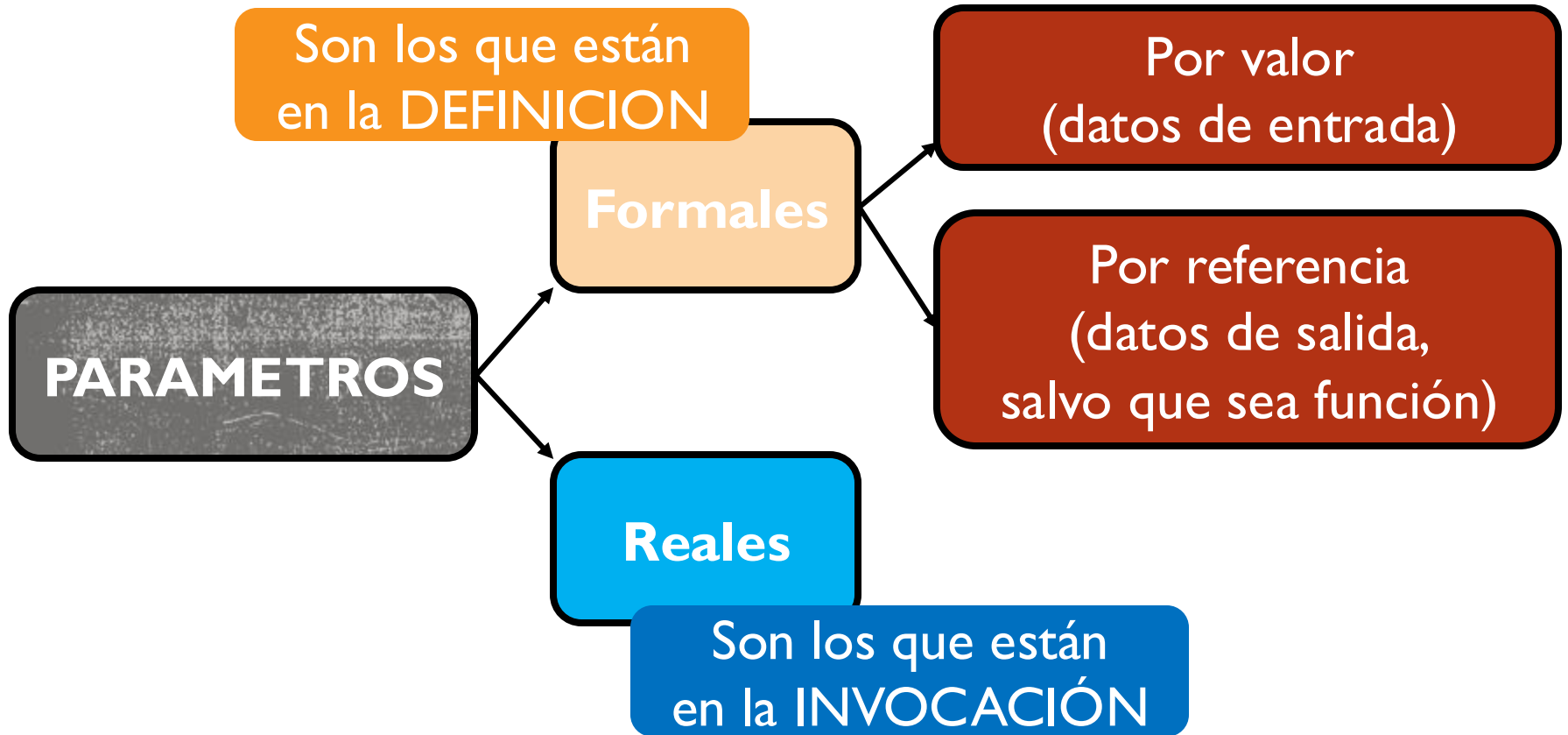
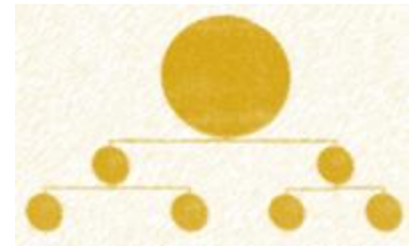
Por valor

Por referencia

Parámetros
formales

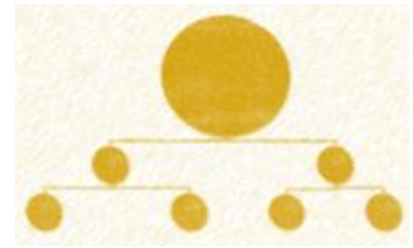


Procedimientos



FUNDAMENTAL ENTENDER LA DIFERENCIA ENTRE **DEFINIR** LA PRIMITIVA (se hace una sola vez antes del cuerpo del programa) e **INVOCAR** (usarla las veces que sea necesario)

Procedimientos



Problema: Dada una secuencia de PARES de números naturales que termina en 0 0, se desea mostrar el mínimo de cada par.

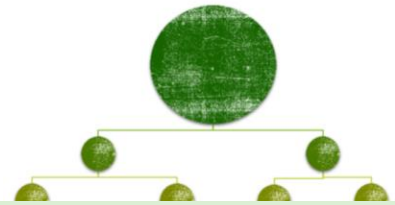
Problema principal: Recorrer completa una secuencia de PARES de números hasta los terminadores.

Sub-problema: Dados dos números naturales, devolver el mínimo.
(ya tenemos hecha la función)

Cuando mandan tarea para la casa //
Pero tú vives en un apartamento



Funciones



Problema: Dados dos números enteros, determinar si ambos tienen igual cantidad de dígitos.

Sub-problema: Contar la cantidad de dígitos de un número entero n .

Algoritmo **cantDigitos**

DE: n {entero}

DS: **cantDigitos** {entero}

Comienzo

$\text{cantDigitos} \leftarrow 0$

mientras $(n \neq 0)$ hacer

$n \leftarrow n \text{ div } 10$

$\text{cantDigitos} \leftarrow \text{cantDigitos} + 1$

Fin

Algoritmo dosNumeros

DE: $n1, n2$ {entero}

DS: igualCD {lógico}

Daux: $cdn1, cdn2$ (natural)

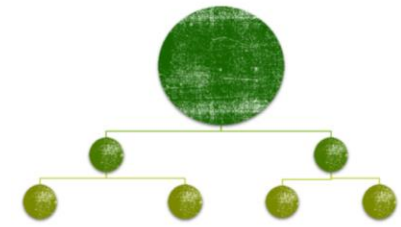
Comienzo

Obtener la cantidad de dígitos de $n1$ y guardarlo en $cdn1$

Obtener la cantidad de dígitos de $n2$ y guardarlo en $cdn2$

$igualCD \leftarrow cdn1 = cdn2$

Fin



Esto será nuestro PROGRAMA

PRINCIPAL

Algoritmo cantDigitos

DE: n {entero}

DS: **cantDigitos** {entero}

Comienzo

$cantDigitos \leftarrow 0$

mientras $(n \neq 0)$ hacer

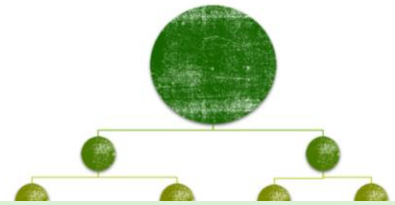
$n \leftarrow n \text{ div } 10$

$cantDigitos \leftarrow cantDigitos + 1$

Fin

Como tenemos UN DATO DE SALIDA,

Funciones



Problema: Dados dos números enteros, determinar si ambos tienen igual cantidad de dígitos.

Sub-problema: Contar la cantidad de dígitos de un número entero.

```
function cantDigitos(n: integer): integer;  
begin  
  cantDigitos:=0;  
  while (n<>0) do  
    Begin  
      n:=n div 10;  
      cantDigitos := cantDigitos + 1  
    End;  
  End;
```

No es lo q yo quiero acá
No quiero invocar!!

Cuando el nombre de una función aparece en una expresión, constituye una **INVOCACION** a la función.



```

9  program Hello;
10 var n: integer;
11
12 begin
13
14     n := 1 + ord;
15
16 end.

```

**MAL INVOCADA
PORQUE LE
FALTAN LOS DATOS
DE ENTRADA**

Compiling main.pas
main.pas(14,17) Fatal: Syntax error, "(" expected but ";" found
Fatal: Compilation aborted
Error: /usr/bin/ppcx64 returned an error exitcode

```

function cantDigitos(n: integer): integer;
begin
cantDigitos:=0;
while (n<>0) do
    Begin
        n:=n div 10;
        cantDigitos := cantDigitos+1
    End;
End;

```

No es lo q yo quiero acá
No quiero invocar!!

Quando el nombre de una función
aparece en una expresión,
constituye una INVOCACION a la función.

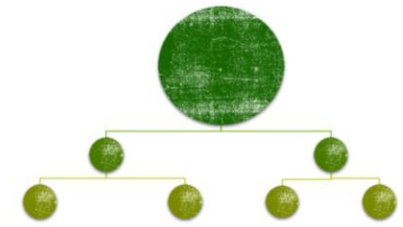
Cuando el nombre de una función aparece en una expresión, constituye siempre una INVOCACION a la función.



```
9  program Hello;
10  var n: integer;
11
12  begin
13
14      n := 1 + ord;
15
16  end.
```

```
Compiling main.pas
main.pas(14,17) Fatal: Syntax error, "(" expected but ";" found
Fatal: Compilation aborted
Error: /usr/bin/ppcx64 returned an error exitcode
```

Funciones



```
function cantDigitos( num : integer ): integer;  
var contador: integer;  
begin  
    contador := 0;  
    while (num <> 0) do  
        begin  
            num := num div 10;  
            contador := contador + 1;  
        end;  
    cantDigitos := contador;  
end;
```

Variable local: almacena temporalmente el dato de salida

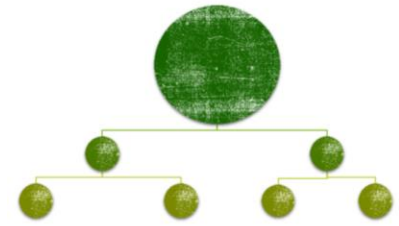
Por ahora, el dato de salida se guarda en una variable local y se asigna al nombre de la función solamente al final de la misma.

Observemos que hemos usado una variable auxiliar para el contador y al terminar el procesamiento de la secuencia asignamos la variable auxiliar al nombre la función.

Funciones

```
program igualCantDigitos;  
var n1, n2, cdn1, cdn2: integer;  
function cantDigitos(num: integer): integer;  
begin
```

*Parámetro
formal*

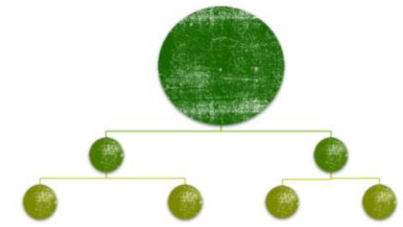


```
    ...  
end;  
begin  
    write ('Ingrese dos números enteros'); readln(n1, n2);  
    cdn1 := cantDigitos(n1);  
    cdn2 := cantDigitos(n2);  
    if (cdn1 = cdn2)  
        then writeln ('Ambos nros tienen = cant de digitos')  
        else writeln ('Los nros no tiene = cant de digitos');  
    end.
```

*Parámetros
reales*

Cuando la función **cantDigitos** termina de ejecutarse,
tanto n1 como n2 conservan su valor

Funciones



*Parámetro
formal*

```
program igualCantDigitos;  
var n1, n2: integer;  
function cantDigitos(num: integer): integer;  
begin
```

...

end;

begin

write ('Ingrese dos números enteros'); readln(n1, n2);

if (**cantDigitos**(n1)=**cantDigitos**(n2))

then writeln ('Ambos nros. tienen = cant. de digitos')

else writeln('Los nros. no tiene = cant. de digitos');

end.

*Parámetros
reales*

Cuando la función **cantDigitos** termina de ejecutarse,
tanto n1 como n2 conservan su valor

¿COMO se **invoca** a una función?

```
function cantDigitos( num : integer): integer;
```

Parámetro
formal
por valor

ENTERO

Maneras de invocar a la función con **distintas opciones para llegar al valor del parámetro real correspondiente a un parámetro por VALOR**

`cant:=cantDigitos(3);` // un literal entero

`cant:=cantDigitos(a);` // una variable de tipo entera

`cant:=cantDigitos(a*10+b);` // una expresión que evalúa a entero

`cant:=cantDigitos(trunc(n));` // una llamada a otra función cuyo resultado es de tipo entero

¡¡Combinaciones de todas las anteriores!!

El parámetro **real** *para los datos de entrada* puede ser cualquier expresión que **compute un valor del tipo del parámetro formal**.

Al momento de la invocación, se evalúa la expresión y el valor obtenido se asigna al parámetro formal.

¿Desde donde se **invoca** a una función?

```
function cantDigitos( num : integer): integer;
```

¿Donde hay **expresiones** en nuestros programas?

- ASIGNACION

dato := **expresion**;

- CONDICIONES

if (**expresion**) then ...

while (**expresion**) do ...

repeat ... until (**expresion**)

- Procedimientos predefinidos write y writeln

writeln('CARTEL', **expresion**)

¿Desde donde se **invoca** a una función?

```
function cantDigitos( num : integer): integer;
```

- ASIGNACION

```
cantDigAyB:=cantDigitos(a) + cantDigitos(b);
```

- CONDICIONES

```
if (cantDigitos(n)>3) then ...
```

```
while (cantDigitos(num)<>0) do ...
```

- Procedimientos predefinidos write y writeln

```
writeln(n, ' tiene ', cantDigitos(n), ' dígitos')
```

```
writeln(cantDigitos(numero)+20)
```

La invocación puede aparecer en todos los lugares donde puede haber un **valor** del **tipo** de salida de la función.

¿Desde donde se **invoca** a una función?

```
function esPrimo(n: integer): boolean;
```

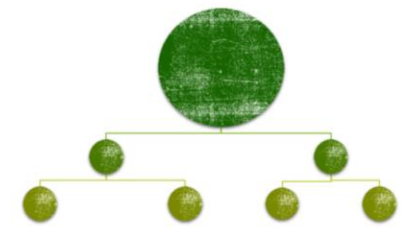
- Asignación

```
cumple := esPrimo(x) and (x < 100);
```

- Condición en **if** o en **while** o en **repeat**

```
if (esPrimo(numero) = TRUE)  
  then ....
```

La invocación puede aparecer en todos los lugares donde puede haber un **valor** del **tipo** de salida de la función.



- ¿Se podría implementar la primitiva que cuenta la cantidad de dígitos de un número, con un procedimiento?
- ¿Cuál de las dos opciones es más adecuada?
- ¿De qué depende la decisión cuando las dos opciones son factibles?



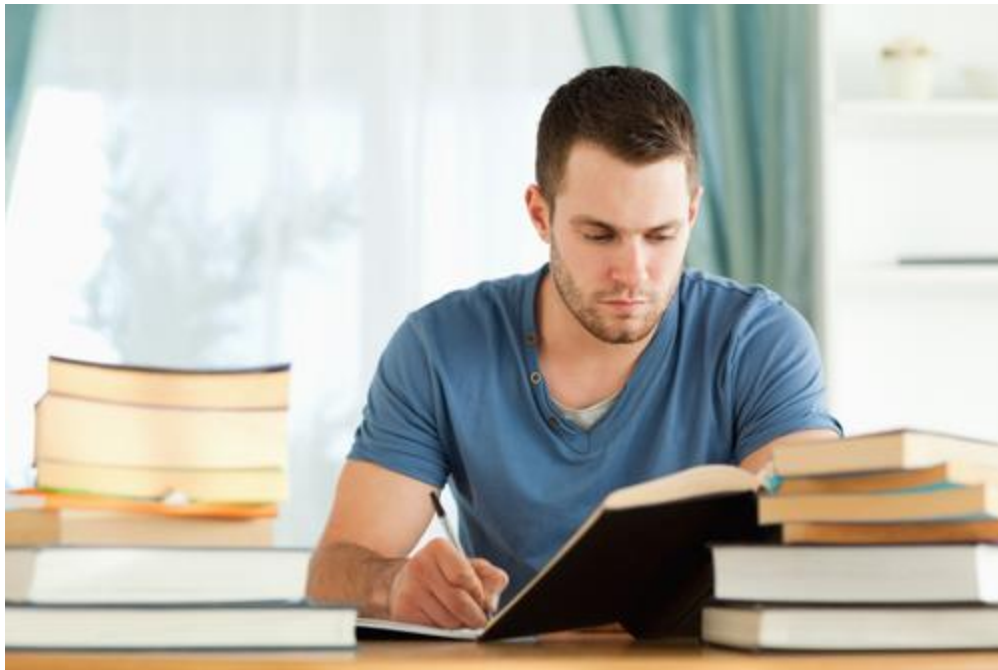
¿¿¿¿FUNCION o
PROCEDIMIENTO?????



¿Función o procedimiento?

- No tiene datos de salida: Procedimiento
- Tiene dos o más datos de salida: Procedimiento
- Tiene un dato de salida de un tipo estructurado: Procedimiento

Tiene un único dato de salida de un tipo simple (número, lógico o char)
REFLEXIONAR SOBRE CÓMO VA A SER USADA LA PRIMITIVA!
Y si tiene efectos colaterales...! Lo veremos más adelante.



**Para estudiar
en casa**

Funciones

Problema 1: Dados 2 caracteres, determinar si al menos 1 de ellos es una vocal minúscula.

Sub-problema: Determinar si un caracter dado es una vocal minúscula.

Algoritmo esVocalMin

DE: car (Char)

DS: esVocalMin (lógico)

Si (car='a') o (car='e') o (car='i') o (car='o') o (car='u')
entonces

esVocalMin $\leftarrow$ VERDADERO

sino

esVocalMin $\leftarrow$ FALSO

Un dato de salida
de un tipo simple
→ FUNCION

FUNCIONES

Sub-problema: Determinar si un caracter dado es una vocal minúscula.

```
function esVocalMin(car:char): boolean;  
begin  
if (car = 'a' or car = 'e' or car = 'i' or car = 'o' or car = 'u')  
then  
  esVocalMin := TRUE;  
end;
```



**Si la letra no es vocal minúscula,
la función no da ningún resultado!!!**

Definición de funciones:

El **nombre** de la función aparece por lo menos dos veces,
en el **encabezamiento (definición)**
y a la **izquierda de al menos una asignación.**

FUNCIONES

Sub-problema: Determinar si un caracter dado es una vocal minúscula.

```
function esVocalMin(car:char): boolean;  
begin  
  if (car = 'a' or car = 'e' or car = 'i' or car = 'o' or car = 'u')  
  then  
    esVocalMin := TRUE  
  else  
    esVocalMin := FALSE;  
end;
```



**EN TODOS LOS POSIBLES CASOS
DEBE DEVOLVER UN RESULTADO**

FUNCIONES

Sub-problema: Determinar si un caracter dado es una vocal minúscula.

```
function esVocalMin(car:char): boolean;
```

```
begin
```

```
  esVocalMin := car = 'a' or car = 'e' or car = 'i'  
               or car = 'o' or car = 'u';
```

```
end;
```

car in ['a', 'e', 'i', 'o', 'u'];

Definición de funciones:

El **nombre** de la función aparece por lo menos dos veces,
en el **encabezamiento (definición)**
y a la **izquierda de al menos una asignación**.

Funciones

Problema: Dados 2 caracteres, determinar si al menos 1 de ellos es una vocal minúscula.

```
program alMenosUno;  
  var ch1, ch2: char;      respuesta: boolean;  
  function esVocalMin(car:char): boolean;  
    ... // Definición de la función  
begin  
  writeln('Ingrese 2 caracteres');  
  readln(ch1, ch2);  
  respuesta := esVocalMin(ch1) OR esVocalMin(ch2);  
  if (respuesta=TRUE) then writeln('Hay al menos una vocal minúscula')  
    else writeln('No hay ninguna vocal minúscula');  
end.
```

FUNCIONES

La variable *car* de tipo *char* es el **parámetro formal** de la función.

Cuando la función se **invoque** debe hacerse con un **parámetro real** de tipo *char*.

La función **computa** un **resultado de tipo *boolean***.

La **invocación** debe aparecer en un contexto que requiera **un valor de tipo *boolean***.

Por ejemplo en la expresión lógica de un *while* o de un *if*.

FUNCIONES

Problema: Determinar si un caracter dado es una letra minúscula.

```
function esMinuscula(car:char) : boolean;  
begin  
    if (car >= 'a') and (car <= 'z')  
    then esMinuscula := true  
    else esMinuscula := false;  
end;
```

FUNCIONES

Problema: Determinar si un caracter dado es una consonante minúscula.

```
function esConsonanteMin(car: char): boolean;  
begin  
  if (esMinuscula(car) and not (esVocalMin(car)))  
  then esConsonanteMin := true  
  else esConsonanteMin:= false;  
end;
```

esConsonanteMin := esMinuscula(car) and not (esVocalMin(car);


```
function esVocalMin(car: char): boolean;  
begin  
  if (car = 'a') or (car = 'e') or (car = 'i') or (car = 'o') or (car = 'u')  
  then esVocalMin := true  
  else esVocalMin := false;  
end;
```

```
function esMinuscula(car: char): boolean;  
begin  
  if (car >= 'a') and (car <= 'z')  
  then esMinuscula := true  
  else esMinuscula := false;  
end;
```

Quedan definidas 3 primitivas:
esVocalMin(char) → boolean
esMinuscula(char) → boolean
esConsonanteMin(char) → boolean

```
function esConsonanteMin(car: char): boolean;  
begin  
  if (esMinuscula(car)) and not (esVocalMin(car))  
  then esConsonanteMin := true  
  else esConsonanteMin := false;  
end;
```

FUNCIONES

Problema 2: Leer una secuencia de caracteres terminada en punto y contar la cantidad de vocales minúsculas, consonantes minúsculas y otros caracteres que contiene la secuencia.

FUNCIONES

```
begin
contv := 0; contc := 0; conto := 0; fin:=false;
write('Ingrese una frase terminada en punto ');
while (fin=false) do
begin
  read(ch);
  if (ch='.')
  then fin:=true
  else
    if esVocalMin(ch)
    then contv:= contv+1
    else
      if esConsonanteMin(ch)
      then contc := contc + 1
      else conto := conto + 1
end;
... // mostrar los resultados
```

FUNCIONES

program secuencia;

...

begin

contv := 0; contc := 0; conto := 0; fin

write('Ingrese una frase terminada en

while (fin=false) do

begin

read(ch);

if (ch='.')

then fin:=true

else

if esVocalMin(ch)

then contv:= contv+1

else

if esConsonanteMin(ch)

then contc := contc + 1

else conto := conto + 1

end;

... // mostrar los resultados

end.

```
function esVocalMin(car: char): boolean;
begin
  if (car = 'a') or (car = 'e') or (car = 'i') or (car = 'o') or (car = 'u')
  then esVocalMin := true
  else esVocalMin := false
end;

function esMinuscula(car: char): boolean;
begin
  if (car >= 'a') and (car <= 'z')
  then esMinuscula := true
  else esMinuscula := false
end;

function esConsonanteMin(car: char): boolean;
begin
  if (esMinuscula(car)) and not (esVocalMin(car))
  then esConsonanteMin := true
  else esConsonanteMin := false
end;
```

FUNCIONES

```
program secuencia;
```

```
var ch: char; fin: boolean;
```

```
    contv, contc, conto: integer;
```

```
...
```

```
begin
```

```
    ...// mostrar los resultados
```

```
    writeln ('Vocales ', contv);
```

```
    writeln ('Consonantes ', contc);
```

```
    writeln ('Otros Caracteres ', conto);
```

```
end.
```